The Zcash Arboretum

Crosslink in Brief

The core ideas of the Crosslink 2 construction

Marek
m@rek.onl

Abstract

This short version of the *Crosslink Guide* states the core of Crosslink 2, the construction of the design book *Zcash Trailing Finality Layer* of N. Wilcox and D.-E. Hopwood, which adds a Byzantine-fault-tolerant finality protocol to Zcash's proof-of-work best chain. It states the problem the construction addresses, its cross-links and validity rules, the two chains each node derives from them, what is proved of their safety and what is only argued of liveness, Stalled Mode, the departures of Shielded Labs' design, the finaliser set, and what "final" means to a client. It gives definitions and statements without proofs; each is cited to the section of the *Crosslink Guide* that treats it and gives the complete rules and sources. The design runs only on Shielded Labs' test network. The document assumes the *Consensus Guide* for work, the best chain and confirmations. It is non-normative: the Zcash Protocol Specification and the ZIPs are authoritative, and neither specifies Crosslink.

Contents

1 Scope and status **4**

2 The availability-finality dilemma **4**

3 The two sub-protocols **6**
3.1 The best-chain protocol 6
3.2 The BFT protocol 7

4 The construction **7**
4.1 The rules 8
4.2 Stalled Mode 10

5 The two chains of a node **10**

6 Safety **11**
6.1 The Prefix Agreement route 11
6.2 The Final Agreement route 12
6.3 Consequences 12

7 Liveness and bounded availability **13**

8 The Shielded Labs design **14**

9 The finaliser set **14**

10 What “final” means to a client **15**

11 Open problems **16**

1 Scope and status

Crosslink 2 is the construction of N. Wilcox and D.-E. Hopwood, *Zcash Trailing Finality Layer*, called *the book* below, whose Crosslink 2 chapters are unchanged since 13 December 2024; the book calls itself "an early and incomplete protocol design proposal". It is a hybrid of Zcash's proof-of-work best chain and a *Byzantine-fault-tolerant* (BFT) protocol: a protocol whose participants decide by votes and which remains safe while fewer than a fixed fraction of them deviate arbitrarily from it. The *Crosslink Guide* gives the construction in full, with every rule, proof and source; this document states its core and cites, for each item, the section of the *Crosslink Guide* that treats it.

As of 26 October 2026, no ZIP specifies Crosslink. Draft Crosslink texts exist in an open, unmerged draft pull request to the ZIP repository, each marked "ZIP: Unassigned"; they are not ZIPs, they weaken or drop several rules of the book, and they are not used here. The design runs only on Shielded Labs' test network, relaunched on 1 October 2026: there is no deployment on Mainnet or Testnet, no activation decision and no production value of any parameter. The network upgrade NU7 (ZIP 259, Proposed) does not include Crosslink. Its 25-second block target spacing (ZIP 218, Proposed) makes every Crosslink parameter counted in blocks cover a third of its earlier duration. As the *Crosslink Guide* records as of 16 October 2026 (§"Scope and status"), no security audit firm has reviewed the design, and no paper on arXiv or in the IACR ePrint archive analyses it.

Classes and status words are those of the *Crosslink Guide*, §"Epistemic classes". A component, that is, a data structure, rule, parameter or check, is *specified* when normative text of the protocol specification or of a ZIP fixes it, *designed but unspecified* when a published design with an analysis exists but no normative text fixes it, and an *open problem* when no design exists. No Crosslink rule is specified: every component of Crosslink is designed but unspecified or an open problem. A result carries one status word: "proved (*S*)" when the source *S* gives a complete proof, *S* being the ebb-and-flow paper of Neu, Tas and Tse (§2), the TFL book or the *Crosslink Guide*; "argued (*S*)" when *S* gives a prose argument only; "assumption"; or "open problem". Every Crosslink result below concerns a designed-but-unspecified construction. A proved result is an implication about a design and makes no component specified.

2 The availability-finality dilemma

Under proof of work, a block gains confirmations as blocks are mined above it, and the probability that it is reversed falls with their number but never reaches zero (*Consensus Guide*, §"Confirmation requirements", Proposition "No finite count is final"). No count of confirmations makes a block final. A BFT protocol decides by a quorum of votes, and its decisions are not reversed while fewer than a fixed fraction of its participants, a third in the protocols considered here, deviate from it; but when too few participants are online, no quorum forms and the protocol stops.

A *ledger* is a sequence of transactions that a node outputs. A ledger protocol is *safe* when the ledgers of honest nodes never conflict, one always being a prefix of the other, and *live* when every transaction given to honest nodes eventually enters the ledger of every honest node. Under *dynamic participation*, honest nodes stop and resume taking part without notice; a *network partition* is an interval during which messages between two sets of honest nodes are not delivered. The

availability-finality dilemma is the statement that no ledger protocol is both live under dynamic participation and safe under network partition: a node cannot tell sleeping peers from partitioned ones, so it either keeps extending its ledger and loses safety under partition, or waits for a quorum and loses liveness when many nodes sleep. The dilemma is used here as a statement, not as a theorem (*Crosslink Guide*, §"The availability-finality dilemma").

J. Neu, E. N. Tas and D. Tse, "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma", resolve the dilemma by giving each node two ledgers: a *finalised* ledger, safe under network partition, and a *dynamically available* ledger, live under dynamic participation, of which the finalised ledger is a prefix. Each client chooses which of the two to trust. Their construction, *snap-and-chat*, runs a longest-chain protocol and a BFT protocol together. A BFT proposal references a block of the longest chain, whose chain is the proposal's *snapshot*; an honest node votes for a proposal only if it sees its snapshot as confirmed in its own longest chain; and the finalised ledger is the transactions of the finalised snapshots in order, with duplicates and transactions invalid at their position removed, a step called sanitisation. The paper proves its security theorem for a permissioned instance: proved (ebb-and-flow paper). Its statement for proof of work is supported by one sentence: argued (ebb-and-flow paper). The paper proves no theorem about Crosslink or about proof of work (*Crosslink Guide*, §"The snap-and-chat construction" and §"The proof-of-work setting").

Remark 2.1 (From snap-and-chat to Crosslink 2). Four of the respects in which the book departs from snap-and-chat determine the construction of Crosslink 2 (*Crosslink Guide*, §"From snap-and-chat to Crosslink 2").

(1) *Target.* The finalised ledger is to remain safe even when the BFT protocol is entirely subverted, provided the adversary holds less than half of the hash rate.

(2) *Checkable confirmation.* A proposal carries the headers of a fixed number of best-chain blocks above its snapshot, so that any party can check from their proof of work that the snapshot is confirmed.

(3) *No sanitisation.* Sanitisation can filter the concatenated transactions of all snapshots, or remove duplicate snapshot blocks first; the two readings coincide when a transaction invalid at a position stays invalid at every later one, as for a double spend or a missing input. A Zcash shielded transaction names an *anchor*, the root of a pool's note commitment tree after some earlier block (*Ironwood Guide*, §"Anchors"), and the same root can arise from unrelated transactions, so a transaction invalid at its position for a missing anchor can become valid at a later one. Both ledgers are therefore plain block chains, and no transaction is reordered or removed.

(4) *Spending finalised outputs.* Validity of a block may depend only on the block and its ancestors, so each best-chain block commits to a BFT block from which its finalised prefix is computable, and that commitment never moves backwards along a chain.

Status: argued (TFL book).

3 The two sub-protocols

Definition 3.1 (Crosslink 2)**.** Crosslink 2 keeps a best-chain protocol Π_{origbc} and a BFT protocol Π_{origbft} as black boxes and transforms them into protocols Π_{bc} and Π_{bft} by structural additions, changed validity rules and changed rules of honest behaviour, so that the blocks of each commit to blocks of the other. These mutual commitments are the *cross-links*. Every node takes part in both protocols.

The blocks, chains and genesis blocks of the two protocols are bc-blocks, bc-chains and $\mathcal{O}_{\text{bc}}$, and bft-blocks, bft-chains and $\mathcal{O}_{\text{bft}}$. A node i is *honest at time t* in an execution if it has followed the protocol up to t and the adversary has not cut it off from the network, for example by an eclipse; an eclipsed node is not honest, and no guarantee below covers it. No synchrony is assumed unless a sub-protocol imports it. The properties below are properties of single executions, and the results are implications between them: when every execution with property $\mathcal{P}$ has property $\mathcal{Q}$, the probability that $\mathcal{Q}$ fails is at most that of $\mathcal{P}$ failing. No probabilistic security theorem in the sense of the ebb-and-flow paper exists for Crosslink 2 (*Crosslink Guide*, §"The hybrid construction", §"Executions, chains and prefix order", §"The best-chain protocol" and §"From snap-and-chat to Crosslink 2").

A block is identified with the chain of which it is the tip, and the *height* $\mathsf{height}(B)$ of a block B is its number of predecessors, so that a genesis block has height 0. For a chain C and $k \in \mathbb{N}$, the chain $C\lceil^k$, with subscript bc or bft where the kind matters, is the prefix of C whose tip has height $\max\{0, \mathsf{height}(C) - k\}$: the chain C without its last k blocks. A chain B is a prefix of C, written $B \preceq C$ (with subscript bc or bft), when B is an initial segment of C, equality included; B and C *agree*, written $B \nsim C$, when one is a prefix of the other, and *conflict* otherwise. Agreement is not transitive: the genesis block agrees with two chains that conflict.

3.1 The best-chain protocol

The view of a node in Π_{origbc} is a set of bc-chains rooted at $\mathcal{O}_{\text{bc}}$; validity of a block depends only on the block and its ancestors; and the best chain ch_i^t of node i at time t is a valid chain of highest score in its view. For Zcash the score is accumulated work and the best chain is the valid chain of most work (*Consensus Guide*, §"Work, and the best chain"). The proof of work of a block is checkable from its header given a valid parent chain.

Definition 3.2 (Prefix Consistency and Prefix Agreement)**.** Let $k \in \mathbb{N}$. An execution of a best-chain protocol has *Prefix Consistency at depth k* iff $\mathsf{ch}_i^t\lceil_{\text{bc}}^k \preceq_{\text{bc}} \mathsf{ch}_j^u$ for all times $t \le u$, every node i honest at t and every node j honest at u, $i = j$ permitted. It has *Prefix Agreement at depth k* iff $\mathsf{ch}_i^t\lceil_{\text{bc}}^k \nsim \mathsf{ch}_j^u\lceil_{\text{bc}}^k$ for all times t and u and the same nodes.

Prefix Consistency at depth k implies Prefix Agreement at depth k. Assuming Prefix Consistency at depth k for proof of work excludes network partitions longer than about k block times: once the best chain on one side of a partition has grown by more than k blocks, its truncation contains blocks that the other side cannot know (*Crosslink Guide*, §"The best-chain protocol").

Remark 3.3 (The prefix properties for Zcash). For Zcash proof of work, Prefix Consistency and Prefix Agreement at any depth are assumptions, not results. No source proves either property for Zcash, and no source computes an adequate depth under the 25-second spacing. Every result below whose premise is a prefix property is therefore conditional for Zcash.

Status: open problem.

3.2 The BFT protocol

An execution of Π_{origbft} proceeds in BFT epochs. In each, a proposer may make a *proposal*, which names its parent bft-block and its BFT epoch and is signed by its proposer; for a proposal P, as for a bft-block, $P\lceil^{1}_{\text{bft}}$ denotes its parent. Each BFT epoch has n *voting units*, distributed among the *validators* of the BFT protocol; the holders of a BFT epoch's units form its *finaliser set*, each being a *finaliser*. A proposal is *notarised* by ballots cast for it by at least $q = \lceil 2n/3 \rceil$ units of its BFT epoch, a protocol being free to require more, and a bft-block is a notarised proposal together with its notarisation proof. By the *Parent rule*, a bft-block other than $\mathcal{O}_{\text{bft}}$ is valid only if its parent is valid.

Definition 3.4 (Finality in context)**.** The protocol supplies an efficiently computable function bft-last-final that maps each valid bft-block C to a prefix $\mathsf{bft\text{-}last\text{-}final}(C) \preceq_{\text{bft}} C$, the *last final block in the context of* C, with $\mathsf{bft\text{-}last\text{-}final}(\mathcal{O}_{\text{bft}}) = \mathcal{O}_{\text{bft}}$ and $\mathsf{bft\text{-}last\text{-}final}(C) \preceq_{\text{bft}} \mathsf{bft\text{-}last\text{-}final}(D)$ whenever $C \preceq_{\text{bft}} D$ are valid. A bft-block B is *final in the context of* C iff $B \preceq_{\text{bft}} \mathsf{bft\text{-}last\text{-}final}(C)$. No bft-block is final other than in a context.

Assumption 3.5 (One-third bound)**.** In the execution considered, for every BFT epoch, strictly fewer than one third of its voting units are ever cast non-honestly. A unit is cast non-honestly when a party other than its holder casts it, for example after a key compromise, when it is cast for two distinct proposals of the BFT epoch, or when its holder casts it against the rules of honest voting.

Status: assumption.

The bound covers all time: a key compromised long after its BFT epoch can still cast that BFT epoch's units.

Definition 3.6 (Final Agreement)**.** An execution of a BFT protocol has *Final Agreement* iff $\mathsf{bft\text{-}last\text{-}final}(C) \preceq \mathsf{bft\text{-}last\text{-}final}(C')$ for all valid bft-blocks C and C' in honest view at any times.

If Assumption 3.5 fails at some time, Final Agreement may fail from that time on. Unlike the prefix properties of proof of work, Final Agreement can hold in an asynchronous network (*Crosslink Guide*, §"The BFT protocol").

4 The construction

Definition 4.1 (Parameters)**.** Crosslink 2 has a *confirmation depth* $\sigma \in \mathbb{N}^{+}$, a *finalisation gap bound* $L \in \mathbb{N}^{+}$, and a predicate is-stalled-block on bc-blocks. No value of σ or L is fixed: the values are an open problem.

The book requires L to be "significantly greater than σ" and "*at least* 2σ" in practice, without derivation; the *Crosslink Guide* proves that $L \geq \sigma + 1$ is necessary for Stalled Mode to end while the BFT protocol is live (§7).

Definition 4.2 (Cross-link fields)**.** Each bc-header carries a field context_bft, a commitment to a bft-block, the *context* of the block. Each proposal and each bft-block B other than $\mathcal{O}_{\text{bft}}$ carries a field B.headers_bc, a sequence of σ bc-headers, deepest first. The *σ-block tail* of a bc-chain T of at least $\sigma + 1$ blocks is the sequence of the headers of $T\lceil^{\sigma-1}, \ldots, T\lceil^{1}, T$.

Definition 4.3 (Derived quantities)**.** For a proposal or bft-block B, the *snapshot* $\mathsf{snapshot}(B)$ is the parent of the deepest header that B carries, and $\mathsf{snapshot}(\mathcal{O}_{\text{bft}}) = \mathcal{O}_{\text{bc}}$; when B carries the σ-block tail of T, $\mathsf{snapshot}(B) = T\lceil^{\sigma}$. For a bc-block H,

$$\begin{aligned} \mathsf{LF}(H) &:= \mathsf{bft\text{-}last\text{-}final}(H.\mathsf{context_bft}), \\ \mathsf{finality\text{-}depth}(H) &:= \mathsf{height}(H) - \mathsf{height}\big(\mathsf{snapshot}(\mathsf{LF}(H))\big), \\ \mathsf{candidate}(H) &:= \mathsf{lca}\big(\mathsf{snapshot}(\mathsf{LF}(H)),\, H\lceil^{\sigma}\big), \end{aligned}$$

where lca is the last common ancestor. The block $\mathsf{LF}(H)$ is the *last final context* of H, and $\mathsf{snapshot}(\mathsf{LF}(H))$ is its *final snapshot*.

A bft-block thus points down into the best chain through its carried headers, and a bc-block points up into the BFT chain through its context (Figure 1; *Crosslink Guide*, §"Parameters" and §"Structural additions"); each node computes its finalised chain from these quantities (§5).

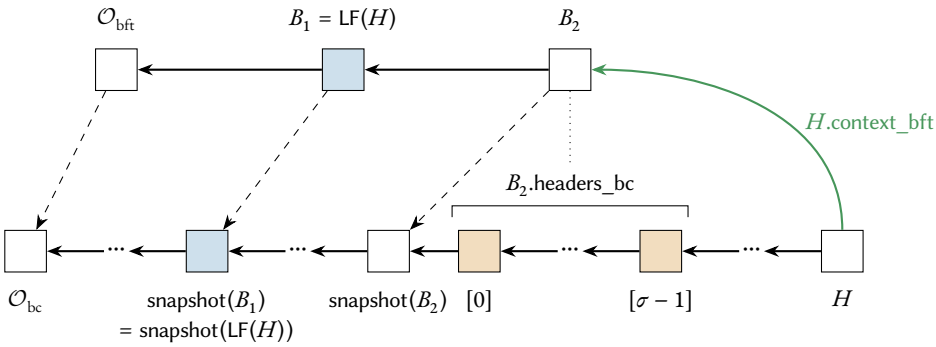


Figure 1. The cross-links. Solid black arrows point from child to parent on the bc-chain (below) and the bft-chain $\mathcal{O}_{\text{bft}}, B_1, B_2$ (above); dashed arrows point from a bft-block to its snapshot. The bracket marks the σ headers B_2.headers_bc[0] to B_2.headers_bc[$\sigma - 1$] (orange) that B_2 carries, and $\mathsf{snapshot}(B_2)$ is the parent of the deepest of them. The green arrow is H.context_bft $= B_2$. In the case drawn, the last block final in the context of B_2 is B_1 (blue), so $\mathsf{LF}(H) = B_1$ and the final snapshot of H is $\mathsf{snapshot}(B_1)$ (blue). The Linearity rule appears as $\mathsf{snapshot}(B_1)$ lying at or below $\mathsf{snapshot}(B_2)$ on one bc-chain.

4.1 The rules

Definition 4.4 (Validity in Π_{bft})**.** A proposal or bft-block B other than $\mathcal{O}_{\text{bft}}$ is valid in Π_{bft} iff it is valid in Π_{origbft} and

(i) *Linearity rule*: $\mathsf{snapshot}(B\lceil^1_{\text{bft}}) \preceq_{\text{bc}} \mathsf{snapshot}(B)$, so that snapshots never move backwards along a bft-chain;

(ii) *Tail Confirmation rule*: B.headers_bc is the σ-block tail of a bc-valid chain.

The function bft-last-final of Π_{bft} is that of Π_{origbft}.

Definition 4.5 (Honest voting)**.** A validator v at time t first adds the bc-blocks of the carried headers of a proposal P to its view, checking their validity, and then votes for P only if P is valid in Π_{bft} and the *Confirmed best-chain criterion* holds:

$$\mathsf{snapshot}(P) \preceq_{\text{bc}} \mathsf{ch}^t_v\lceil^\sigma,$$

that is, the snapshot is σ-confirmed in the validator's own best chain.

Definition 4.6 (Validity in Π_{bc})**.** The genesis block has $\mathcal{O}_{\text{bc}}$.context_bft = $\mathcal{O}_{\text{bft}}$. A bc-block $H \neq \mathcal{O}_{\text{bc}}$ is valid in Π_{bc} iff it is valid in Π_{origbc} and

(i) *Valid context rule*: H.context_bft is a valid bft-block;

(ii) *Extension rule*: $\mathsf{LF}(H\lceil^1_{\text{bc}}) \preceq_{\text{bft}} \mathsf{LF}(H)$, so that the last final context never moves backwards along a bc-chain;

(iii) *Last Final Snapshot rule*: $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\text{bc}} H$, so that the final snapshot lies on the block's own chain;

(iv) *Finality depth rule*: $\mathsf{finality\text{-}depth}(H) \leq L$ or $\mathsf{is\text{-}stalled\text{-}block}(H)$.

The rules on transactions are those of Π_{origbc}, unchanged.

Definition 4.7 (Unchanged fork choice)**.** An honest node uses no information from Π_{bft} other than the validity rules of Definition 4.6 to choose the bc-chain it follows: ch^t_i is a bc-valid chain of highest score in its view, as in Π_{origbc}.

The BFT protocol steers the best chain only through validity: a chain whose blocks cite contexts that move finality backwards, or place the final snapshot off the chain, is invalid, and the most-work rule chooses among the rest. The rules answer items 2 to 4 of Remark 2.1: the Tail Confirmation rule makes confirmation checkable by any party (item 2), while the Confirmed best-chain criterion keeps snap-and-chat's voting condition; the Linearity rule makes the snapshots of a valid bft-chain extend one another, so that the finalised history is one bc-chain and needs no sanitisation (item 3); and the Extension and Last Final Snapshot rules make the finalised prefix computable from a block's own chain (item 4). The Linearity rule also carries the safety argument of §6.2, and the Finality depth rule bounds availability (§4.2). All of these definitions are designed but unspecified (*Crosslink Guide*, §"Validity rules of the BFT protocol" to §"Honest block production and

fork choice").

4.2 Stalled Mode

Definition 4.8 (Stalled Mode)**.** A bc-block H is in *Stalled Mode* iff $\mathsf{finality\text{-}depth}(H) > L$. By the Finality depth rule, every valid bc-block in Stalled Mode is *stalled*: it satisfies is-stalled-block.

For Zcash the book proposes that a stalled block be *coinbase-only*: it has exactly one transaction, which only distributes newly issued funds and has no inputs; the book never fixes the predicate. Stalled Mode is an objective property of each block, not a network-wide switch. In Stalled Mode the best chain can grow only by stalled blocks, which under the proposed policy carry no ordinary spends, and that it keeps growing is argued (TFL book) (§7); a block that cites a context whose final snapshot lies on its chain at most L blocks below it is not in Stalled Mode, whatever its parent (*Crosslink Guide*, §"Stalled Mode").

5 The two chains of a node

From its best chain, each node derives two chains, which play the roles of the finalised and the available ledger of the ebb-and-flow model (*Crosslink Guide*, §"The locally finalised chain" and §"The locally bounded-available chain").

Definition 5.1 (Locally finalised chain)**.** Each node i keeps a bc-chain fin_i^t, starting at $\mathsf{fin}_i^0 = \mathcal{O}_{\mathrm{bc}}$. At each time t at which its best chain changes, with s the time of the previous change, or 0 if there is none,

$$\mathsf{fin}_i^t := \begin{cases} \mathsf{candidate}(\mathsf{ch}_i^t), & \text{if } \mathsf{fin}_i^s \preceq_{\mathrm{bc}} \mathsf{candidate}(\mathsf{ch}_i^t), \\ \mathsf{fin}_i^s, & \text{otherwise;} \end{cases}$$

if $\mathsf{candidate}(\mathsf{ch}_i^t)$ conflicts with fin_i^s, the node records a *finalisation safety hazard*. Between changes of its best chain, fin_i is constant.

Lemma 5.2 (Local finalization linearity)**.** *A node that follows the update rule of Definition 5.1 has* $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{fin}_i^u$ *for all* $t \le u$*: its finalised chain never rolls back.*

Status: proved (Crosslink Guide).

The chain fin_i^t is state of the node, not a field of any block, and no consensus rule reads it. A BFT decision alone changes nothing at a node. For a node i honest at t, a bc-block $X \neq \mathcal{O}_{\mathrm{bc}}$ lies in fin_i^t exactly when four conditions hold: X lies in the snapshot of a proposal; the bft-block formed from that proposal is final in the context of some valid bft-block; some valid bc-block H with $X \preceq_{\mathrm{bc}} H\lceil^{\sigma}$ has a last final context that extends that bft-block; and H has become the node's best tip, by time t, at an update that set fin_i to $\mathsf{candidate}(H)$ (*Crosslink Guide*, §"Finality as a status", Proposition "Path to finality"). When $\mathsf{LF}(H) \neq \mathcal{O}_{\mathrm{bft}}$ and the headers carried by $\mathsf{LF}(H)$ are ancestors of H, $\mathsf{candidate}(H)$ is the final snapshot of H, so an update to it leaves fin_i trailing the tip by $\mathsf{finality\text{-}depth}(H) \ge \sigma + 1$ blocks; the bound is attained (*Crosslink Guide*, §"Computation of the finalised prefix", Proposition "Finalisation lag").

Definition 5.3 (Locally bounded-available chain)**.** Each node chooses a depth μ with $0 < \mu \leq \sigma$, by default $\mu = \sigma$. Its *locally bounded-available chain* at time t is

$$(\mathsf{ba}_\mu)_i^t := \begin{cases} \mathsf{ch}_i^t\lceil^\mu, & \text{if } \mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^t\lceil^\mu, \\ \mathsf{fin}_i^t, & \text{otherwise.} \end{cases}$$

Proposition 5.4 (Ledger prefix property)**.** *For every node i honest at time t, $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} (\mathsf{ba}_\mu)_i^t$.*
Status: proved (TFL book).

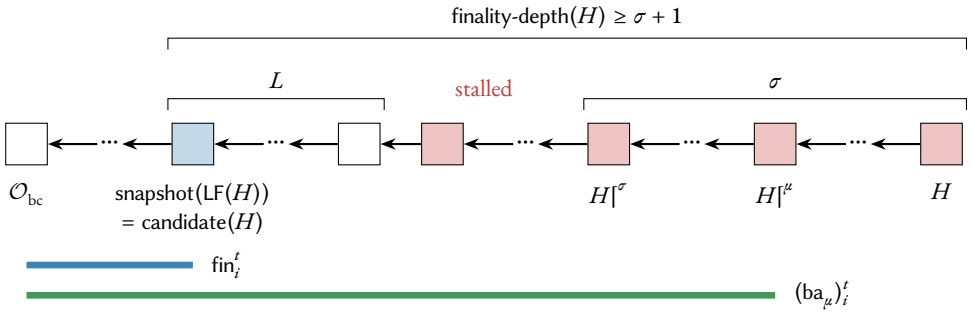


Figure 2. The two chains of node i at an update with best tip $H = \mathsf{ch}_i^t$ in Stalled Mode, when the headers that $\mathsf{LF}(H)$ carries are ancestors of H and the update sets fin_i^t to $\mathsf{candidate}(H)$. Arrows point from child to parent. The final snapshot $\mathsf{snapshot}(\mathsf{LF}(H))$ is the lower of itself and $H\lceil^\sigma$, hence $\mathsf{candidate}(H)$; fin_i^t (blue) ends there, $\mathsf{finality\text{-}depth}(H) \geq \sigma + 1$ blocks below H. The chain $(\mathsf{ba}_\mu)_i^t$ (green) is $H\lceil^\mu$, and fin_i^t is a prefix of it. The blocks more than L above the final snapshot, up to H, are stalled (red). Positions are symbolic and represent no value of σ, μ or L.

6 Safety

Definition 6.1 (Assured Finality)**.** An execution has *Assured Finality* iff $\mathsf{fin}_i^t \not\bowtie \mathsf{fin}_j^u$ for all times t and u, every node i honest at t and every node j honest at u, $i = j$ permitted.

Assured Finality is the book's main safety goal: the finalised chains of honest nodes may differ in length, but never conflict (*Crosslink Guide*, §"Assured Finality"). Two routes lead to it, one through each sub-protocol.

6.1 The Prefix Agreement route

Theorem 6.2 (Prefix Agreement implies Assured Finality)**.** *In every execution of Crosslink 2 in which Π_{bc} has Prefix Agreement at depth σ, Assured Finality holds.*
Status: proved (TFL book).

Corollary 6.3 (Safety under subversion of the BFT protocol)**.** *In every execution in which* Π_{bc} *has Prefix Agreement at* σ*, Assured Finality holds whatever the holders of voting units do, including executions in which every voting unit is cast non-honestly and signatures are forged.*
Status: proved (Crosslink Guide).

The proof of Theorem 6.2 uses nothing of the BFT protocol: each finalised chain of a node is a prefix of one of its earlier best chains truncated by σ, and Prefix Agreement makes those truncations agree. This meets the target of item 1 of Remark 2.1, conditionally on Prefix Agreement of Π_{bc}, whose transfer from Π_{origbc} to Π_{bc} is open (*Crosslink Guide*, §"The Prefix Agreement route").

6.2 The Final Agreement route

Theorem 6.4 (Final Agreement implies Assured Finality)**.** *In every execution of Crosslink 2 in which* Π_{bft} *has Final Agreement, Assured Finality holds.*
Status: proved (Crosslink Guide).

The book states this theorem, but its printed proof is a copy of its proof of Theorem 6.2; the *Crosslink Guide* gives a proof, which uses neither σ nor Prefix Agreement but needs the Linearity rule (*Crosslink Guide*, §"The Final Agreement route").

Proposition 6.5 (Necessity of the Linearity rule)**.** *Let the Linearity rule be removed. For every* σ *there is an execution in which* Π_{bft} *has Final Agreement, the best chains of two honest nodes* i *and* j *satisfy the Valid context, Extension and Last Final Snapshot rules, the voters of every bft-block they cite meet the Confirmed best-chain criterion, and* fin_i *and* fin_j *conflict.*
Status: proved (Crosslink Guide).

6.3 Consequences

Corollary 6.6 (Assured Finality under either premise)**.** *Assured Finality holds in every execution in which* Π_{bc} *has Prefix Agreement at* σ *or* Π_{bft} *has Final Agreement.*
Status: proved (Crosslink Guide).

The finalised chains of two honest nodes can therefore conflict only in an execution in which both premises fail. A partition of the network, which breaks Prefix Agreement, together with an adversary holding more than a third of the units of a BFT epoch and voting with each side, which breaks Final Agreement, is such an execution; the book calls this attack unavoidable, since no best-chain protocol keeps finality under partition (argued (TFL book); *Crosslink Guide*, §"Consequences of the two routes"). For Zcash, both premises are open: Prefix Agreement of proof of work is an assumption (Remark 3.3), and no concrete BFT protocol, hence no proof of its Final Agreement, is fixed; that Final Agreement of Π_{origbft} carries over to Π_{bft} is argued (TFL book) (*Crosslink Guide*, §"Liveness of the sub-protocols", Proposition "Transfer by erasure"). Crosslink is thus not proved safe for Zcash; it is proved safe under either of two premises, each of which is open for Zcash.

The bounded-available chain is less safe than the finalised chain. Agreement of the chains ba_μ of honest nodes follows from Prefix Agreement at depth μ (proved (Crosslink Guide)), and not from Final Agreement (*Crosslink Guide*, §"Safety of the bounded-available chain").

7 Liveness and bounded availability

No liveness statement of Crosslink 2 is proved. The book argues that the best chain of Π_{bc} grows whenever that of Π_{origbc} does, given that a coinbase-only stalled block can always be produced; that Π_{bft} is live whenever Π_{origbft} is; and that Stalled Mode is not entered while there are enough honest block producers, proposers and validators and L is large enough: argued (TFL book), each. The transfer of the prefix properties from Π_{origbc} to Π_{bc}, the advance of finalisation and a rule for choosing L are open problems. Three proved results bound what any liveness argument can claim (*Crosslink Guide*, §"Liveness of the sub-protocols", §"Progress of finalisation" and §"Stalled Mode"):

(i) if $L \le \sigma$, Stalled Mode persists even while Π_{bft} is live, so $L \ge \sigma + 1$ is necessary;

(ii) block producers that never advance context_bft over L consecutive heights force a stalled block whatever Π_{bft} does;

(iii) holders of more than one third of the units of a BFT epoch who withhold their votes, and whose units no one else casts, prevent every notarisation of that BFT epoch.

Bounded availability is the book's informal weakening of dynamic availability: during a finalisation stall, the number of best-chain blocks above the final snapshot of the best tip may grow only up to a bound, beyond which ordinary spends stop. The book argues for it that under unbounded availability the cost of rolling back spent funds grows without limit while finality stalls, and that halting the chain instead would give an attacker unbounded time to build a fork of the recent tail: argued (TFL book) (*Crosslink Guide*, §"Bounded availability"). The book does not prove the bound; the *Crosslink Guide* proves it for a node whose finalised chain lies on its best chain.

> **Proposition 7.1** (Bounded non-stalled run)**.** *Let $H = \mathsf{ch}_i^t$, and let node i have followed the update rule of Definition 5.1 up to t. Suppose that the Extension, Last Final Snapshot and Finality depth rules hold along H, that the Parent and Linearity rules hold along the bft-chain that ends at $\mathsf{LF}(H)$, that $\sigma \le L$, and that $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} H$. Then every block of H at height greater than $\mathsf{height}(\mathsf{fin}_i^t) + L$ is stalled.*
>
> *Status: proved (Crosslink Guide).*

For some best-chain protocol with the properties of §3.1 the constraint $\sigma \le L$ cannot be dropped (proved (Crosslink Guide)); the case in which fin_i^t is not a prefix of the best chain is open (*Crosslink Guide*, §"The bounded non-stalled run"). A *finality override*, a deliberate reversal of finalised blocks in response to a failure of the protocol, has no rule, message or state in the construction: it is a change of the consensus rules, a hard fork outside the protocol, and Assured Finality binds executions of the protocol only. The book's case for formalising overrides is argued (TFL book) (*Crosslink Guide*, §"Finality overrides").

8 The Shielded Labs design

The design that Shielded Labs is building, described in its *Zcash Crosslink Design Overview* (draft 10, 14 September 2026), keeps the Tail Confirmation, Linearity, Valid context, Extension and Last Final Snapshot rules and the update rule of fin. It drops the Finality depth rule, Stalled Mode, the bound L and the chain ba_{μ}. It adds a *sticky fork choice*, under which a node follows the heaviest valid chain that contains its finalised chain; an *Inclusion Depth rule*, a lower bound $\mathsf{finality\text{-}depth}(H) \geq \sigma + 1$, not an upper bound; the BFT protocol Tenderlink, derived from Tendermint and without a specification, whose bft-blocks, called *certificates*, carry no signatures: a best-chain block cites a certificate through a reference that carries its notarisation signatures; and *slash forks*, hard forks that node operators may adopt to punish named finalisers (§9). Every component is designed but unspecified (*Crosslink Guide*, §"The Shielded Labs design").

Under its rules, both routes to Assured Finality are proved (Crosslink Guide), but neither premise is: Final Agreement of Tenderlink is an open problem, and Prefix Agreement of the best chain under sticky fork choice is unproved, while one formal model exhibits its failure on a small instance in which more than a third of the BFT voting power is subverted. Sticky fork choice never records a finalisation hazard (proved (Crosslink Guide)); its safety and liveness in general are open (*Crosslink Guide*, §"Sticky fork choice"). Without the Finality depth rule, the run of best-chain blocks above fin is unbounded: for every d there is an execution in which the best chain extends d blocks beyond the finalised chain (proved (Crosslink Guide); *Crosslink Guide*, §"Theorems under the Shielded Labs design").

9 The finaliser set

The book models only n voting units per BFT epoch and a two-thirds threshold, and lists the selection of a proof-of-stake sub-protocol and the rewards of stakers as missing. Every concrete element of the finaliser set below is Shielded Labs' design and is designed but unspecified, with the open problems noted (*Crosslink Guide*, §"The finaliser set").

- *Voting power.* The finalisers are the participants of Tenderlink. A finaliser's voting power is the stake delegated to it; the N finalisers of largest power form the active roster, and a certificate needs two thirds of the active roster's stake. Every fraction is a fraction of stake, never of the number of finalisers. The value of N and the selection rule are open.

- *Roster.* The roster of each BFT height, the height of a certificate in the bft-chain, derives from the staking state at the snapshot of the certificate decided at the preceding BFT height. That snapshot generally lies above a node's finalised chain and may lie off its best chain; the roster cannot change while no new certificate is decided, except by a slash fork.

- *Delegation.* A staker locks shielded ZEC into a bond, whose amount is a power of ten ZEC, delegated to one finaliser and redelegable at any time. Leaving takes two steps, unbonding and withdrawing, in different *staking windows*, recurring ranges of best-chain heights to which the creation of bonds, unbonding and withdrawal are confined, so that misbehaviour can be punished before the funds leave.

- *Privacy.* Batching in staking windows and amounts quantised at creation are meant to keep an observer from linking a bond to the shielded notes that funded it. The totals of each finaliser and the amount and target of each bond are public; hiding the delegation itself is an open problem.
- *Rewards.* Staking is paid from the existing block subsidy, with no new issuance and no change to the distribution of fees. The published split gives 40 per cent of the subsidy to miners, 40 per cent to stakers and 20 per cent to Zcash's development fund, which is not a Crosslink share; of the stakers' part, 90 per cent goes to bonds and 10 per cent to finalisers as commission. Whether payment depends on new finality is open.
- *Slashing.* No rule of the protocol punishes a finaliser. Node operators may instead adopt a slash fork, a hard fork that burns the bonds delegated to named finalisers and removes those finalisers from the roster.

Proposition 9.1 (Power of a finaliser coalition)**.** *When ballots cannot be forged and notarisation proofs are sound, holders of more than one third of a BFT epoch's units can prevent every notarisation of that BFT epoch by withholding. In every execution, whatever the holders of voting units do, they cannot make the finalised chains of honest nodes conflict while* Π_{bc} *has Prefix Agreement at* σ*.*

Status: proved (Crosslink Guide).

10 What "final" means to a client

Definition 10.1 (Final)**.** A bc-block X is *final at node i at time t* iff $X \preceq_{\mathrm{bc}} \mathsf{fin}_i^t$. A transaction is final at i at t iff a block that contains it is final at i at t.

This is the book's assured finality: the protocol cannot revert a final transaction, while a fork made outside the protocol, a finality override among them, still can. Finality of a bc-block is relative to one node, and differs from finality of a bft-block in a context (Definition 3.4; *Crosslink Guide*, §"Finality as a status").

Proposition 10.2 (Guarantee of finality)**.** *Let X be final at node i at time t, with i honest at t. Then X is final at i at every later time at which i is honest. In every execution in which* Π_{bc} *has Prefix Agreement at* σ *or* Π_{bft} *has Final Agreement, moreover, X and* fin_j^u *agree for every time u and every node j honest at u.*

Status: proved (Crosslink Guide).

At each time, a block is in exactly one of three states at a node: final; not yet final, when it strictly extends the node's finalised chain; or never final, when it conflicts with it. The state is decided by ancestry, not by height, and a block not yet final has no promise of becoming final (*Crosslink Guide*, §"Finality as a status", Proposition "Finality status").

Finality is distinct from confirmation depth. A deep block outside fin_i^t is not final, and a block of fin_i^t is final at any depth; confirmation counts still measure proof-of-work depth. A block that

becomes final at an update under the hypotheses of the lag bound of §5 has at least $\sigma + 2$ confirmations at that time (proved (Crosslink Guide); *Crosslink Guide*, §"Finality and confirmation depth").

An application reads one of two views, fin_i^t or $(\mathsf{ba}_\mu)_i^t$, never the best chain itself. The choice sets its behaviour during a finalisation stall: with fin it stops at once; with ba_μ it keeps processing user transactions until L blocks have passed. Shielded Labs' design has neither ba_μ nor L: an application there either stops with fin or follows the best chain, which can run unboundedly ahead of it (*Crosslink Guide*, §"Ledger views and stalls"). A light client that verifies block headers only cannot verify finality in any current design: no design places in the header chain both the notarisation signatures and a commitment to the finalisers against which they are counted. This is an open problem (*Crosslink Guide*, §"Light clients").

11 Open problems

A specification of Crosslink 2 for Zcash must fix each of the following components, and no source fixes any of them (*Crosslink Guide*, §"Components a specification must fix"): the concrete BFT protocol, the assignment of its voting units and the format of its notarisation proofs; the values of σ and L, with $\sigma + 1 \leq L$; the predicate is-stalled-block; the encoding and placement of the cross-link fields; the mapping from stake to voting units and the selection of the roster; the staking calendar; the format of bond transactions, the quantised amounts and a minimum stake; the reward rule and the division of the commission; the fees of staking transactions; the rotation of finaliser keys and the initial roster; and activation on the existing chain. Four partial formal models exist, in Lean, Quint and TLA$^+$; none machine-checks Assured Finality or either theorem of §6 (*Crosslink Guide*, §"Coverage of the formal models").

Table 1. Status by goal (*Crosslink Guide*, §"Status by goal and by rule").

Goal	Result	Status
fin_i never rolls back	Lemma 5.2	proved (Crosslink Guide)
$\mathsf{fin}_i^t \preceq_{\mathrm{bc}} (\mathsf{ba}_\mu)_i^t$	Proposition 5.4	proved (TFL book)
Assured Finality	Theorem 6.2, Theorem 6.4, Corollary 6.6	proved (TFL book); proved (Crosslink Guide)
Safety with the BFT protocol subverted	Corollary 6.3	proved (Crosslink Guide)
Bounded availability	Proposition 7.1, finalised chain on the best chain	proved (Crosslink Guide); otherwise open problem
Liveness	§7	argued (TFL book)
The premises for Zcash	Remark 3.3; no BFT protocol fixed	open problem